A MINIMAL FIELD HANDBOOK

The Minimal Distributed Training Handbook

DDP, ZeRO/FSDP, tensor and pipeline parallelism, communication and scaling plans.

Mosafer

DDP · FSDP · Megatron · torchrun

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **2**
- 1.1 The memory math that starts everything . . . 2
- 1.2 The parallelism menu . . . 2
- 1.3 The efficiency metric: MFU . . . 2

2 Data Parallelism **4**
- 2.1 DDP in one script . . . 4
- 2.2 The communication math . . . 4
- 2.3 Batch size and the ladder . . . 5

3 ZeRO and FSDP **6**
- 3.1 The three stages . . . 6
- 3.2 FSDP in one script . . . 6
- 3.3 Choosing . . . 6

4 Tensor and Pipeline Parallelism **8**
- 4.1 Tensor parallelism . . . 8
- 4.2 Pipeline parallelism . . . 8
- 4.3 Composing: 3-D parallelism . . . 8

5 Communication and Overlap **10**
- 5.1 The collectives . . . 10
- 5.2 The bandwidth arithmetic . . . 10
- 5.3 Overlap: hiding the tax . . . 10

6 Training at Scale **12**
- 6.1 Precision at scale . . . 12
- 6.2 Checkpointing at scale . . . 12
- 6.3 Loss spikes: the ops manual . . . 12

7 Scaling Plans **14**
- 7.1 The ladders . . . 14
- 7.2 A planning worksheet . . . 14
- 7.3 Frameworks . . . 14

A Cheat Sheet **16**
- A.1 Memory arithmetic . . . 16
- A.2 Strategy table . . . 16
- A.3 Communication . . . 16
- A.4 Launch and resume . . . 16

CHAPTER 1

Orientation

Distributing training answers one question: your model does not fit on one GPU (or your deadline does not). The answer space is small — split the data, split the model, or both — and this handbook is the decision map plus the arithmetic that decides it.

1.1 The memory math that starts everything

A model in training occupies four memory pools:

Pool	Size at bf16 with AdamW
weights	$2N$ bytes
gradients	$2N$ bytes
Adam moments (m, v)	$4N$ bytes (fp32)
master weights (fp32 copy)	$4N$ bytes
activations	scales with batch × context × depth — checkpointable (PyTorch handbook, Ch. 12)

Total: $\approx 16N$ bytes + activations. A 7B model needs ≈ 112 GB before activations — no single card. *This* is why sharding exists, before any throughput argument.

1.2 The parallelism menu

Strategy	Splits	Communication
data (DDP)	the batch	gradients, once per step
ZeRO/FSDP	weights, grads, moments + batch	parameters and gradients, per step
tensor (TP)	each layer's matrices	activations, several times per layer
pipeline (PP)	layers into stages	activations at stage borders
sequence/context	the sequence axis	per attention chunk

Real large runs compose them: FSDP over nodes, tensor parallel inside a node, data parallel over everything (Ch. 6's 3-D parallelism).

1.3 The efficiency metric: MFU

Model FLOPs Utilization = achieved FLOPs/s divided by peak. A 4090 peaks at 165 TFLOPS bf16; a training run at 60 TFLOPS is 36% MFU — good. MFU is the honest cross-cluster comparison and the number that tells you whether an optimization matters:

```
tokens_per_step = 256 * 1024
N = 7e9
step_flops = 6 * N * tokens_per_step
mfu = step_flops / (step_time_s * 8 * 165e12)   # 8 GPUs x peak
# FLOPs per token for a transformer: ~6N (Ch. 6, DL theory handbook)
```

PITFALL

The scaling-law warning (DL theory handbook, Ch. 6) applies to infrastructure: a distributed setup that changes the *effective* batch or data order changes the experiment. Ch. 2–5 change where computation runs, never what it computes — verify with a one-GPU-vs-multi-GPU equivalence test on a small model before trusting a large one.

CHAPTER 2

Data Parallelism

Data parallelism is the default: every GPU holds the full model, sees a different shard of data, and gradients are averaged once per step. Simple, scalable — up to the memory wall (Ch. 1's $16N$).

2.1 DDP in one script

The PyTorch handbook (Ch. 12) has the working DDP boilerplate; the essentials restated:

```
import os, torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler

def setup():
    dist.init_process_group("nccl")           # torchrun provides the env
    torch.cuda.set_device(int(os.environ["LOCAL_RANK"]))

setup()
rank, world = dist.get_rank(), dist.get_world_size()
model = model.to(rank)
ddp = DDP(model)                              # broadcast weights on wrap
sampler = DistributedSampler(data, shuffle=True)   # disjoint shards

for epoch in range(E):
    sampler.set_epoch(epoch)                  # reshuffle per epoch!
    for batch in loader:
        loss = step(ddp, batch)               # grads all-reduced in backward
        if rank == 0:                         # one writer only
            log(loss)
```

The mechanism: DDP `backward()` overlaps gradient all-reduce with backprop (bucketed by reverse layer order) — by the time the last gradient is computed, earlier ones are already averaged.

2.2 The communication math

All-reduce of $2N$ bytes of gradients per step over a ring costs $\approx 2N \cdot \frac{2(w-1)}{w}$ bytes of wire per GPU — effectively $4N$ bytes at large world size. On PCIe at 25 GB/s, a 7B model: ≈ 1.1s per step *just communicating* — compute is 6ND FLOPs; if compute per step is smaller than that, you are communication-bound:

Interconnect	Bandwidth	7B-model all-reduce
PCIe 4.0 x16	~25 GB/s	~1.1 s/step
NVLink (4090 lacks it; A100/H100 have it)	300–900 GB/s	0.03–0.09 s/step
InfiniBand (between nodes)	200–400 Gb/s per NIC	nodes are the slow edges

The rule that falls out: tensor parallel *inside* a node (fast links), data parallel *across* nodes (slow links, one all-reduce per step).

2.3 Batch size and the ladder

DDP's effective batch = per-GPU batch × world size. The linear-scaling rule (Optimization handbook, Ch. 6) applies up to the critical batch size; warmup scales with it. Ladder up: single GPU \to DDP 2–8 \to + gradient accumulation \to FSDP when $16N$ exceeds one card (Ch. 3).

PITFALL

DDP traps: (1) forgetting `sampler.set_epoch()` — identical data order every epoch, silent; (2) saving `ddp_model.state_dict()` — `module.`-prefixed keys that will not load (PyTorch handbook, Ch. 12); (3) logging on every rank — duplicated tensorboard runs and file races; (4) unbalanced batches with `drop_last=False` — one straggling rank stalls the all-reduce on every step.

THE ONE-SENTENCE SUMMARY

DDP is one all-reduce of gradients per step, overlapped with backprop — the first tool, correct until $16N$ bytes + activations exceed one card, at which point Chapter 3 shards the pools.

CHAPTER 3

ZeRO and FSDP

ZeRO (Zero Redundancy Optimizer) removes DDP's redundancy: instead of every GPU holding every weight, gradient, and optimizer moment, each GPU stores a *shard*. FSDP is PyTorch's built-in ZeRO-3.

3.1 The three stages

Stage	Sharded	Memory per GPU (vs. DDP)
ZeRO-1	optimizer moments	$\approx 16N/w + 4N$
ZeRO-2	+ gradients	$\approx 16N/w + 2N$
ZeRO-3 / FSDP	+ parameters too	$\approx 16N/w$+ activations

with w = world size. ZeRO-3 makes a 70B model trainable on 8x40 GB: per-GPU parameter memory drops to $2N/w$ — parameters are gathered *just in time* for each layer's forward, then released.

3.2 FSDP in one script

```
from torch.distributed.fsdp import (FullyShardedDataParallel as FSDP,
    ShardingStrategy, MixedPrecision, CPUOffloadPolicy)

fsdp = FSDP(model.to(rank),
    sharding_strategy=ShardingStrategy.FULL_SHARD,   # ZeRO-3
    mixed_precision=MixedPrecision(
        param_dtype=torch.bfloat16,
        reduce_dtype=torch.float32),
    auto_wrap_policy=transformer_auto_wrap_policy,   # shard per block
    device_id=rank)
```

Mechanics per layer: all-gather the layer's parameters (compute), free them after use, reduce-scatter gradients — communication doubles relative to DDP (parameters down + gradients back), which is why `auto_wrap_policy` shards per transformer block (gather granularity) and why prefetching the next block matters.

```
torchrun --nproc_per_node=8 train.py   # the DDP loop (Ch. 2), FSDP swapped in
```

3.3 Choosing

Situation	Choice
model fits one card	DDP — lowest communication
$16N$ exceeds one card	FSDP FULL_SHARD (+ activation checkpointing)
gradient accumulate at scale	FSDP + `use_orig_params` for PEFT compat
7B on 8x24 GB	ZeRO-2 often suffices; ZeRO-3 if activations squeeze

PITFALL

FSDP traps: (1) checkpointing — `torch.save(fsdp.state_dict())` saves sharded state; use FSDP's `state_dict_type=FULL_STATE_DICT` on rank 0 (or the distributed checkpointing API) or your checkpoint loads nowhere; (2) PEFT + FSDP needs `use_orig_params=True` and the right `param_dtype` or LoRA adapters train in the wrong precision; (3) tiny models under FSDP are slower than DDP — the all-gather overhead dominates; there is a minimum useful size.

DEEPSPEED EQUIVALENCE

`deepspeed` implements the same stages (ZeRO-1/2/3) with its own API and offload-to-CPU/NVMe extras (ZeRO-Offload/Infinity). FSDP is PyTorch-native and the default recommendation; DeepSpeed wins on offloading and on Megatron-DeepSpeed pipelines. The concepts are identical — stages are stages.

CHAPTER 4

Tensor and Pipeline Parallelism

FSDP shards parameters; tensor and pipeline parallelism split the *computation*. TP cuts each layer across GPUs (keeping activations local to fast links); PP cuts the layer stack into stages.

4.1 Tensor parallelism

Megatron-style splitting of a transformer block:

- **column-parallel** Linear: $W = [W_1; W_2]$ — each GPU computes its slice of the output; no communication until the next op needs the full vector.
- **row-parallel** Linear: $W = [W_1|W_2]$ — inputs are split, outputs are *all-reduced*.
- the transformer pattern: attention and MLP1 column-parallel, MLP2 row-parallel — one all-reduce per sublayer boundary; LayerNorm stays replicated.
- **Megatron-style marking** (frameworks infer it from the model definition):
- QKV projection *o* `ColumnParallelLinear`
- attention out *o* `RowParallelLinear`
- MLP up → `ColumnParallelLinear`; MLP down → `RowParallelLinear`

Communication: 2 all-reduces per transformer block *per forward* (and again in backward) — per token, per layer. This is why TP demands NVLink: the traffic scales with tokens × layers, not with steps.

4.2 Pipeline parallelism

Split layers into stages; GPU i holds stages and passes activations forward, gradients backward. The naive schedule idles (bubbles):

Schedule	Bubble fraction
naive	$(p-1)/(m+p-1)$ — with p stages, m micro-batches
1F1B (one-forward-one-backward)	same bound, far less activation memory
interleaved 1F1B	smaller bubbles by splitting stages across GPUs

The fix for bubbles is *micro-batching*: split the global batch into m micro-batches so all stages stay busy simultaneously — $m \geq 4p$ keeps bubbles under 20%.

```
from torch.distributed.pipelining import pipeline, ScheduleGPipe
pipe = pipeline(model, mb_args=(micro_batch_shape,))
schedule = ScheduleGPipe(pipe, n_microbatches=8)
# torch.distributed.pipelining (PyTorch 2.4+):
```

4.3 Composing: 3-D parallelism

Large runs stack: TP=4 within a node (NVLink), PP=2 across two nodes, data parallel 16 over the rest — total world 128. The assignment rule: fastest communication (TP) on the fastest links; FSDP/DP across the slow ones; PP to fill the middle. Frameworks (`Megatron-LM`, `DeepSpeed`, `torchtitan`) express this as three integers and handle the collectives.

PITFALL

Two composition traps: (1) TP + FSDP on the same axis double-shard and thrash — TP is for *within*-node, FSDP for *across*-node; (2) PP changes the effective batch — each micro-batch flows through all stages per step, so `global_batch = micro_batch * m * data_parallel`: the Optimization handbook's batch-size arithmetic now has three factors, and the lr scaling must follow all of them.

THE DECISION RULE

Start with FSDP + data parallel (Ch. 3). Add tensor parallel only when a single layer's activations or weights cannot fit the gather window (very large models, very long context). Add pipeline parallel when the model spans nodes. Every added axis buys memory at the price of communication and complexity — compose in that order.

CHAPTER 5

Communication and Overlap

Collectives are the tax of distribution. This chapter is the small library of collectives, the bandwidth arithmetic, and the overlap tricks that turn tax into pipeline.

5.1 The collectives

Collective	Semantics	Used by
all-reduce	sum (or avg) a tensor across ranks; everyone gets the result	DDP gradients
reduce-scatter	sum and shard — each rank gets a slice	FSDP gradient sync
all-gather	assemble shards — everyone gets the whole	FSDP parameter gather
broadcast	one rank \to all	weight sync on wrap
barrier	wait until all ranks arrive	phase boundaries

```
dist.all_reduce(t, op=dist.ReduceOp.SUM); t /= world
dist.all_gather_with_grad(shards, local_param)   # FSDP-style gather
```

5.2 The bandwidth arithmetic

Ring all-reduce of B bytes over w ranks moves $2B(w-1)/w$ bytes per rank — essentially $2B$ for large w. Per step at world 8, bf16 7B gradients: ≈ 14 GB per rank. The budget check that decides your parallelism layout (Ch. 1's table, Ch. 2's rule):

$$\text{comm time per step} \;\geq\; \frac{2N \cdot \text{bytes}}{\text{bandwidth}} \quad \text{— overlap it or eat it.}$$

5.3 Overlap: hiding the tax

- **bucketing**: all-reduce gradients in reverse-layer buckets so communication runs while later layers still compute (DDP does this; tune `bucket_cap_mb`).
- **prefetch**: FSDP all-gathers layer i+1's parameters while layer i computes.
- **compression**: bf16 gradient reduce instead of fp32 (halves traffic); quantized all-reduce is an active research area.
- **overlap windows**: TP's all-reduces are the reason TP lives on NVLink — on slow links, switch the axis, do not tune.

```
ddp = DDP(model, bucket_cap_mb=25)      # default 25 MB; larger = fewer
                                        # syncs, more memory
# DDP bucket tuning:
```

PITFALL

Three communication traps: (1) `torch.cuda.synchronize()` sprinkled for debugging left in production code — it serializes the overlap; (2) CPU-side control flow between ranks diverging (an `if rank ==`

`0` around a *collective* call) — one rank calls all-reduce, another does not, and the job hangs at NCCL timeout (default 30 min of silence); (3) measuring comm with wall-clock instead of `nccl` profilers — overlap makes wall-clock lie in both directions.

THE ONE-SENTENCE SUMMARY

Every distributed step pays a collective bill sized in bytes — DDP pays it once per step and overlaps it; FSDP pays per layer and prefetches; TP pays per layer per token and demands NVLink. The layouts of Chapters 2–4 are just different payment plans.

CHAPTER 6

Training at Scale

The mechanics are Chapters 2–5; operating a large run adds three disciplines: precision strategy, checkpointing at scale, and surviving loss spikes.

6.1 Precision at scale

Component	Standard practice
weights (compute)	bf16 — range of fp32, no scaler needed
master weights	fp32 copies (FSDP/DeepSpeed keep them) — the update accumulator
optimizer moments	fp32, sharded (ZeRO-1)
gradients (comm)	bf16 reduce with fp32 accumulation
embeddings / norms	fp32 copies where the paper says so — the finicky bits

6.2 Checkpointing at scale

A 70B FSDP run's checkpoint is hundreds of gigabytes, sharded across ranks — and the job *will* be preempted. The requirements:

- **distributed checkpoints**: each rank writes its shard (`torch.distributed.checkpoint`); reload onto a *different* world size (elastic resume).
- **frequency vs. cost**: full checkpoint every `save_interval` steps + cheap in-memory replicas for crash-restart.
- **async save**: stage to CPU/NVMe off the critical path (FSDP's `async_save`) — a 5-minute synchronous pause every 1000 steps is a 5% throughput tax.
- **everything in the checkpoint**: weights, optimizer, dataloader state, lr state, RNG states, step counter — the resume must be bitwise-continuous (Git handbook Ch. 7's config discipline applies).

6.3 Loss spikes: the ops manual

Large runs *will* show loss spikes. The playbook, in order:

1. Isolate: is it one rank (bug), one data shard (poison), or global (numerics)?
2. Skip: roll back to the last good checkpoint and skip the offending data batch — the standard LLM-run move.
3. Reduce: lower lr temporarily (the schedule's reverse — Optimization handbook, Ch. 6), raise the batch.
4. Record: spike step, data ids, gradient norms — spikes that repeat are bugs; spikes that do not are noise to plan around.

Track gradient norms per layer continuously — a spike's *location* (one layer vs. all) is the diagnosis.

> **PITFALL**
>
> Three scale-run traps: (1) stale dataloader state after resume — duplicated or skipped data silently shifts the run; (2) TF32 on by default differing between GPUs — bitwise differences across node types are expected; document the fleet; (3) NCCL timeouts tuned too tight — a straggler during preemption windows kills the whole job; set `NCCL_TIMEOUT` generously and checkpoint before long waits.

THE ONE-SENTENCE SUMMARY

At scale, the training loop is 90% unchanged code and 10% new discipline: sharded fp32 masters, async distributed checkpoints, spike playbooks, and per-layer gradient norms — the run that survives a week is the one whose failure modes were rehearsed.

CHAPTER 7

Scaling Plans

From one GPU to a cluster: the concrete ladders. Each rung states the hardware, the strategy, and the limit that forces the next rung.

7.1 The ladders

Hardware	Strategy
1x24 GB	bf16 + gradient checkpointing + accumulation (PyTorch handbook Ch. 6–12)
1x80 GB (A100/H100)	DDP is moot; FSDP for activation headroom; 7B–14B full FT
4–8x80 GB, one node	FSDP full shard; 14B–34B full FT; 70B with activation checkpointing + offload
2–4 nodes	FSDP across nodes (InfiniBand); or Megatron TP=4/PP=2 for 70B+
clusters	3-D: FSDP + TP-in-node + DP; torchtitan/Megatron recipes

7.2 A planning worksheet

1. N and tokens from the scaling law (DL theory handbook, Ch. 6); compute $C = 6ND$ FLOPs.
2. Memory: $16N$ bytes + activations — pick the sharding stage (Ch. 3).
3. Throughput: peak FLOPs × MFU (0.35–0.5 realistic) — step time and run duration.
4. Communication: TP within nodes; the Ch. 5 budget check on inter-node links.
5. Cost: GPU-hours = run duration × GPUs; add 20% for restarts (it happens).

```
N, D = 7e9, 1.4e11
C = 6 * N * D                        # ~5.9e21 FLOPs
mfu, peak = 0.45, 312e12             # A100 bf16 peak per GPU
gpus = 8
seconds = C / (mfu * peak * gpus)
print(f"{seconds/3600:.1f} GPU-cluster-hours")   # ~330 h -> ~1.4 days
# the plan for a 7B / 140B-token run on 8xA100-80GB:
```

7.3 Frameworks

Framework	When
`torchtitan`	PyTorch-native large-scale reference (FSDP2, TP, PP)
`Megatron-LM`	NVIDIA's maximal-throughput recipes (TP+PP+SP)
`DeepSpeed`	ZeRO + offload + easy adoption
`accelerate`	the gentle wrapper (HF handbook, Ch. 8) — small-to-mid scale

PITFALL

Planning traps: (1) quoting peak FLOPs as throughput — a 40% MFU plan is the honest one; (2) forgetting the data pipeline — a run that out-eats its dataloader is a 5%-MFU run with extra steps

(num_workers at scale, dataset sharding across ranks); (3) no dry run — always execute the full config at 1/100 scale for one full cycle (train, checkpoint, resume, eval) before the real launch (PyTorch handbook, Ch. 7's self-test, at cluster scale).

THE ONE-SENTENCE SUMMARY

Pick the cheapest parallelism that fits memory (DDP `\to` FSDP `\to` +TP `\to` +PP), verify equivalence at small scale, size the run with the 6ND arithmetic at honest MFU, and rehearse checkpoint-resume before the expensive launch. That is a scaling plan.

APPENDIX A

Cheat Sheet

A.1 Memory arithmetic

Pool	Bytes
weights + grads + Adam + master	$\approx 16N$ (bf16/fp32 mix)
activations	batch × context × layers — checkpoint to trade
KV (inference)	$2 \times L \times d_{\text{model}} \times$ layers (Inference handbook Ch. 2)

A.2 Strategy table

Need	Strategy
default	DDP (`torchrun`, `DistributedSampler`)
model too big	FSDP FULL_SHARD + activation checkpointing
biggest models	FSDP + TP-in-node + PP-across (Megatron/torchtitan)
offload	DeepSpeed ZeRO-Offload/Infinity
LoRA at scale	FSDP with `use_orig_params=True`

A.3 Communication

Collective	Used by
all-reduce	DDP gradient sync (bucketed, overlapped)
reduce-scatter / all-gather	FSDP shard sync, per layer
broadcast	initial weight sync
TP all-reduces	2 per transformer block, per token — NVLink only

A.4 Launch and resume

```
torchrun --nproc_per_node=8 train.py
NCCL_TIMEOUT=1800 torchrun ...          # generous straggler window
torch.distributed.checkpoint            # elastic, resharding saves
```

Discipline	Detail
checkpoint	weights + optimizer + loader + RNG + step; async save
resume	bitwise-continuous; different world size via DCP
spikes	isolate, skip batch, drop lr, log gradient norms
MFU	achieved/peak FLOPs; 0.35–0.5 is healthy; the comparison metric

THE HABITS THAT MATTER

Verify equivalence at small scale before large launches. TP inside nodes, data/FSDP across. Async checkpoints, rehearsed resumes. Log per-layer gradient norms. Plan with honest MFU. Every distributed failure in this book's chapters was one of these five habits, skipped.